Ders 26 Çalışma Özeti

Ders 26 Çalışma Özeti

Genel Konular

- Dersin tarihi 27 Aralık 2021 Pazartesi. Ders Zoom üzerinden yapıldı.
  - Asıl ders yaklaşık 00:10'da başlar. 00:58-01:11 arası teneffüstür.
  - Slaytlar Silberschatz, *Operating System Concepts* (9. baskı) 7. bölüm: ölümcül kilitlenme (deadlock).
- Sistem modeli: kaynaklar ve kaynak kullanımı (istek, kullanım, bırakma)
- Deadlock'un dört gerekli koşulu
- Kaynak atama çizgesi (resource-allocation graph, RAG)
- Deadlock ile başa çıkma yöntemleri
  - Önleme (prevention), kaçınma (avoidance), tespit ve kurtarma (detection and recovery), görmezden gelme.
- Kaçınma: güvenli durum (safe state), RAG algoritması, banker algoritması
- Tespit: bekleme çizgesi (wait-for graph) ve çok kopyalı kaynaklar için tespit algoritması
- Kurtarma: süreç sonlandırma, kurban seçimi, açlık

Hocanın Özellikle Vurguladığı Kısımlar

- Deadlock için dört koşulun dördü birden sağlanmalıdır. Biri yoksa deadlock olmaz.
  - Önleme yöntemleri bu koşullardan birini engellemeye dayanır: "En iyi savunma saldırıdır."
- Muteks yanlış kullanılırsa deadlock oluşabilir. Muteksi doğru kullanmak programcının sorumluluğudur.
  - İki thread aynı iki kilidi ters sırayla alırsa deadlock olabilir. Ama her çalıştırmada olmayabilir; çok thread'li programlarda hata ayıklamayı zorlaştıran da budur.
- RAG'de döngü yoksa deadlock kesin yoktur. Döngü varsa deadlock olabilir ama kesin değildir.
  - Döngüdeki kaynakların hepsi tek kopyalıysa döngü deadlock demektir.
- Modern işletim sistemleri (Windows, Linux) deadlock için genelde bir şey yapmaz. Çözüm kullanıcıya ve programcıya kalır.
- Banker algoritması her istekte sistemi yeniden kontrol eder. Bu yüzden maliyetlidir ve doğrudan uygulanması zordur.
- Ödev gibi düşünün: Banka transferi fonksiyonunu deadlock'a yol açmayacak şekilde nasıl yazarsınız?

Kısa Tekrar Notları

- Kaynak kullanımı: istek (request), kullanım (use), bırakma (release).
- Dört koşul:
  - Karşılıklı dışlama (mutual exclusion): en az bir kaynak paylaşılamaz.
  - Tut ve bekle (hold and wait): süreç bir kaynağı tutarken başka kaynak bekler.
  - Kesme yok (no preemption): kaynak, süreçten zorla alınamaz.
  - Döngüsel bekleme (circular wait): $P_0 \to P_1 \to \ldots \to P_n \to P_0$ bekleme zinciri.
- RAG: süreç daire, kaynak kare. $P_i \to R_j$ istek kenarı, $R_j \to P_i$ atama kenarı. Kaynağın kopyaları karenin içinde nokta.
- Önleme:

  - Tut ve bekle: süreç bütün kaynaklarını başta ister ya da elinde kaynak yokken ister. Kullanım verimi düşer, açlık olabilir.
  - Kesme yok: istek karşılanmazsa süreç tuttuğu kaynakları bırakır.
  - Döngüsel bekleme: kaynaklar numaralanır, süreçler artan sırayla ister.
- Güvenli durum: bütün süreçlerin bitebileceği bir çalışma sırası (safe sequence) vardır. Güvenli durumda deadlock olmaz.
- Banker algoritması veri yapıları: `Available`, `Max`, `Allocation`, `Need` ($Need = Max - Allocation$).
- Tek kopyalı kaynaklarda tespit: bekleme çizgesinde döngü aramak, $O(n^2)$.
- Çok kopyalı kaynaklarda tespit: banker'a benzer algoritma, $O(m \cdot n^2)$.
- Kurban seçimi ölçütleri: öncelik, kullanılan CPU süresi, işin ne kadarının tamamlandığı, tuttuğu kaynaklar, sonlandırılacak süreç sayısı, etkileşimli ya da batch olması.

Detaylı Açıklamalar

Kaynaklar ve dört koşul

Kaynak fiziksel olabilir: CPU zamanı, bellek, G/Ç aygıtları, yazıcı, USB aygıtı. Mantıksal da olabilir: dosya, semafor, muteks. Süreç bir kaynağı önce ister, sonra kullanır, sonra bırakır. Örneğin `malloc` ile alınan bellek `free` ile bırakılır.

- Karşılıklı dışlama: Okuma için açılan bir dosya paylaşılabilir. Yazma için açılan dosya ya da muteks paylaşılamaz.
- Kesme yok: Burada süreç kesmesi değil, kaynağın geri alınması kastediliyor. Süreç dosya kopyalarken kaynak dosyayı açık tutar, hedef dosyayı bekler. Beklerken açık dosyayı bırakmaz.
- Döngüsel bekleme: Numaraların sıralı olması gerekmez. Önemli olan kümede kapalı bir bekleme zincirinin olmasıdır.

Muteks örneği: Thread 1 önce `first_mutex`, sonra `second_mutex` alır. Thread 2 önce `second_mutex`, sonra `first_mutex` alır. Thread 1 birinci kilidi alıp kesilir, Thread 2 ikinci kilidi alır. Artık ikisi de birbirini bekler. Dört koşulun hepsi sağlanır.

Kaynak atama çizgesi

Slayttaki ilk örnekte P1, P2, P3 süreçleri ve R1-R4 kaynakları var. R2'nin 2, R4'ün 3 kopyası var. Çizgede döngü yok. P3 çalışıp biter, R3'ü bırakır. P2 çalışıp biter, R1'i bırakır. Sonra P1 çalışır. Deadlock yoktur.

İkinci örnekte P3 de R2'yi ister. İki döngü oluşur:

- $P_1 \to R_1 \to P_2 \to R_3 \to P_3 \to R_2 \to P_1$
- $P_2 \to R_3 \to P_3 \to R_2 \to P_2$

R2'nin iki kopyası da P1 ve P2'dedir. Hiçbir süreç ilerleyemez; deadlock vardır.

Üçüncü örnekte $P_1 \to R_1 \to P_3 \to R_2 \to P_1$ döngüsü vardır ama deadlock yoktur. R1'in bir kopyası P2'de, R2'nin bir kopyası P4'tedir. P2 ya da P4 bitince kaynak boşalır ve döngü çözülür.

Deadlock ile başa çıkma

1. Önleme: dört koşuldan birini sistem düzeyinde imkânsız kılmak.
2. Kaçınma: süreçlerden ek bilgi (en fazla kaynak ihtiyacı) alıp, sistemi güvensiz duruma sokacak istekleri ertelemek.
3. Tespit ve kurtarma: deadlock'a izin verip olunca bulmak ve çözmek.
4. Görmezden gelme: Modern işletim sistemleri genelde bunu yapar.

Önleme yöntemleri:

- Karşılıklı dışlama: Paylaşılabilir kaynaklarda sorun yoktur. Ama muteks gibi kaynaklar doğası gereği paylaşılamaz; bu koşul kaldırılamaz.
- Tut ve bekle: CD-ROM'dan okuyup diske yazan, sonra yazıcıya gönderen bir süreç düşünün.

  - Bütün kaynaklar başta alınırsa, yazıcı daha kullanılmadan meşgul edilir.
  - Ya da süreç elinde kaynak yokken istek yapar: önce CD-ROM ve dosyayı alıp bırakır, sonra dosya ve yazıcıyı ister.
  - İki yöntem de verimsizdir ve açlığa yol açabilir.
- Kesme yok: İsteği karşılanmayan süreç tuttuğu kaynakları da bırakır ve baştan ister. İşinin sonuna gelmiş bir sürecin bütün hesabı boşa gidebilir.
- Döngüsel bekleme: Her kaynağa bir numara verilir. Süreçler kaynakları yalnız artan numara sırasıyla ister. Ek yük getirir, programcının sıraya uyması gerekir, sistemden sisteme taşınabilirlik zordur.

Güvenli durum ve kaçınma

Kaçınmada her süreç her kaynaktan en fazla kaç tane isteyeceğini önceden bildirir. Sistem durumu (resource-allocation state), mevcut kaynaklar, atanmış kaynaklar ve en fazla ihtiyaçlardan oluşur.

Sistem şu anki durumdan, süreçleri bir sırayla çalıştırıp hepsini bitirebiliyorsa güvenli durumdadır. Güvenli durumda deadlock olmaz. Güvensiz durum her zaman deadlock demek değildir, ama deadlock'a gidebilir.

RAG algoritması (tek kopyalı kaynaklar): Talep kenarı (claim edge) eklenir: $P_i \dashrightarrow R_j$, kesikli ok. Sürecin ileride bu kaynağı isteyebileceğini gösterir. İstek yapılınca kesikli ok düz ok olur, atanınca yönü değişir. Bir istek, atama kenarına dönüşünce çizgede döngü oluşacaksa karşılanmaz.

Slayttaki örnekte R1 P1'de, R2 P2'de. P2, R1'i istiyor. P1'in R2'ye talep kenarı var. Durum güvensizdir. P1 R2'yi isterse döngü kapanır ve deadlock olur. Bu yüzden bu istek karşılanmaz.

Banker algoritması (çok kopyalı kaynaklar): Adını, bütün müşterilerinin nakit ihtiyacını karşılayabilmek zorunda olan bankalardan alır.

Güvenlik algoritması:

1. `Work = Available`, bütün i için `Finish[i] = false`.
2. `Finish[i] == false` ve `Need[i]` $\leq$ `Work` olan bir i bul. Yoksa 4. adıma geç.
3. `Work = Work + Allocation[i]`, `Finish[i] = true`. 2. adıma dön.
4. Bütün i için `Finish[i] == true` ise durum güvenlidir.

İstek algoritması: P_i `Request[i]` ile gelir.

1. `Request[i]` $\leq$ `Need[i]` değilse hata: süreç bildirdiğinden fazlasını istiyor.
2. `Request[i]` $\leq$ `Available` değilse süreç bekler.
3. İstek karşılanmış gibi yapılar güncellenir: `Available -= Request`, `Allocation += Request`, `Need -= Request`. Yeni durum güvenliyse istek karşılanır. Değilse eski duruma dönülür ve süreç bekler.

Slayttaki örnek (A: 10, B: 5, C: 7 kopya):

Süreç	Allocation	Max	Need
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Available = (3, 3, 2). Güvenli sıra ⟨P1, P3, P4, P2, P0⟩. Hocanın derste bulduğu ⟨P1, P3, P4, P0, P2⟩ sırası da güvenlidir. İkisi de Python ile doğrulandı.

- P1 (1, 0, 2) isterse: Need ve Available'dan küçüktür. Yeni Available = (2, 3, 0). Durum güvenlidir, istek karşılanır.
- Ardından P4 (3, 3, 0) isterse: A'dan yalnız 2 tane vardır. Süreç bekler.
- Ardından P0 (0, 2, 0) isterse: Kaynak vardır. Ama yeni durum (Available = (2, 1, 0)) güvensizdir. İstek karşılanmaz, P0 bekler.

Tespit

Her kaynaktan tek kopya varsa RAG, bekleme çizgesine çevrilir. Kaynak düğümleri kaldırılır. $P_i \to R \to P_j$ yolu, $P_i \to P_j$ kenarı olur: P_i, P_j'yi bekliyor. Bekleme çizgesinde döngü varsa deadlock vardır.

Slayttaki örneğin bekleme çizgesi kenarları: $P_1 \to P_2$, $P_2 \to P_3$, $P_2 \to P_4$, $P_2 \to P_5$, $P_3 \to P_4$, $P_4 \to P_1$. Döngüler $P_1 \to P_2 \to P_4 \to P_1$ ve $P_1 \to P_2 \to P_3 \to P_4 \to P_1$. Kenarlar slayt karesinden okundu.

Çok kopyalı kaynaklar için banker'a benzer bir algoritma kullanılır. `Need` yerine `Request` matrisi vardır. Elinde hiç kaynak olmayan süreç baştan bitmiş sayılır.

Slayttaki örnek (A: 7, B: 2, C: 6 kopya, Available = (0, 0, 0)):

Süreç	Allocation	Request
P0	0 1 0	0 0 0
P1	2 0 0	2 0 2
P2	3 0 3	0 0 0
P3	2 1 1	1 0 0
P4	0 0 2	0 0 2

⟨P0, P2, P3, P1, P4⟩ sırasıyla hepsi biter; deadlock yoktur. P2 ek olarak C'den 1 tane isterse (Request = (0, 0, 1)) yalnız P0 bitebilir. P1, P2, P3 ve P4 deadlock'tadır. Python ile doğrulandı.

Tespit algoritması ne sıklıkla çalışmalı? Her istekte çalışırsa ek yük çok büyür. Belirli aralıklarla çalışırsa, iki çalıştırma arasında birden çok istek olur. O zaman deadlock'a hangi isteğin yol açtığı bilinemez.

Kurtarma

- Deadlock'taki bütün süreçleri sonlandırmak ya da döngü kırılana kadar birer birer sonlandırmak.
- Kaynak geri alma (resource preemption) ile bir kurban seçilir. Kurban seçimi maliyeti en aza indirmeye çalışır. Ölçütler: öncelik, harcanan CPU süresi, işin tamamlanma oranı, tuttuğu kaynaklar, sonlandırılacak süreç sayısı, etkileşimli ya da batch olması.
- Kurban, deadlock öncesindeki güvenli bir duruma geri alınmalıdır (rollback).
- Hep aynı süreç kurban seçilirse açlık oluşur.

Bu sorunlar yüzünden modern işletim sistemleri işi kullanıcıya bırakır.

Banka transferi sorusunun çözümü

Derste verilen fonksiyon önce `from` hesabının, sonra `to` hesabının kilidini alır. A→B transferi ile B→A transferi aynı anda çalışırsa deadlock olabilir. Döngüsel beklemeyi önlemenin yolu kilitleri her zaman aynı sırada almaktır. Örneğin hesap numarası küçük olanın kilidi önce alınır:

```
void transaction(Account from, Account to, double amount) {
    mutex *ilk  = (from.id < to.id) ? get_lock(from) : get_lock(to);
    mutex *ikinci = (from.id < to.id) ? get_lock(to) : get_lock(from);
    acquire(ilk);
    acquire(ikinci);
    withdraw(from, amount);
    deposit(to, amount);
    release(ikinci);
    release(ilk);
}
```

Bu çözüm derste verilmedi; hocanın sorduğu soru için eklendi.

Ders sonrası notlar

- Hoca bir yerde “safe state değilse zaten deadlock olmaz” dedi. Doğrusu: güvenli durumdaysa deadlock olmaz. Güvensiz durumda deadlock olabilir.
- Kaçınma örneğinde “deadlock’a girmesi için P2’nin R2’yi istemesi gerekir” dendi. R2 zaten P2’dedir. Deadlock’a yol açacak istek, P1’in R2’yi istemesidir. Hoca hemen sonra bunu doğru söyledi.
- P0’ın (0, 2, 0) isteği için “büyük ihtimalle deadlock olacak” dendi. Banker algoritması deadlock’tan önce davranır: Yeni durum güvensiz çıktığı için istek reddedilir, P0 bekletilir.
- Tespit örneğinde P2’nin isteği (0, 0, 1) olunca deadlock’taki süreçler P1, P2, P3 ve P4’tür. Derste P1 sayılmadı. Python ile doğrulandı: P1’in isteği (2, 0, 2) de eldeki (0, 1, 0) ile karşılanamaz.
- Çok kopyalı kaynaklarda tespit algoritmasının karmaşıklığı $O(m \cdot n^2)$’dir (m kaynak türü, n süreç). Derste “$n \cdot m^2$” dendi.
- Hoca “Witness” adlı kilit sırası denetleyicisinin Linux’ta olduğunu söyledi. Witness büyük olasılıkla FreeBSD’nin aracıdır. Linux’taki benzer araç büyük olasılıkla `lockdep`’tir.
- “Bütün kaynaklar tek kopyalıysa döngü deadlock demektir” ifadesi için yalnız döngüdeki kaynakların tek kopyalı olması yeterlidir.
- Bekleme çizgesi derste karışık anlatıldı (“R3’ten P5’e” gibi). Doğru kenarlar yukarıdaki “Tespit” kısmında, slayt karesine göre verildi.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.